

Support Chat — API Guide

Guest ↔ Admin messaging, for frontend integration. Covers every endpoint, its request/response shape, auth, and the notification/real-time behaviour behind it.

Update: every support-chat endpoint — guest and admin — now lives on a single controller: `/api/v1/support-chat/*`. Nothing runs through `/vip-app/*` anymore.

Base path —
`/api/v1`

Auth — Bearer JWT (guest or admin token)

Format — all responses wrapped in the standard `ApiResponse` envelope

01 How it works

Every guest has exactly **one conversation** with support — not a list of separate tickets. Think of **Conversation** as the chat's summary card (status, last message, unread count) and **Message** as each individual line in that chat.

Guest side — reads/writes through `/support-chat/my/*`, using the guest's own login token. No conversation id needed — the backend resolves the guest's single conversation automatically.

Admin side — reads/writes through `/support-chat/*`, and every call after the list needs the `conversationId` returned by the list endpoint.

02 Response envelope

Every endpoint below returns this same wrapper — check `success` first, then read `data`.

```
// ApiResponse<T>
{
  "success": true,
  "message": "Message sent",
  "data": { /* T — shape shown per endpoint below */ },
  "errors": [],
```

```
"timestamp": "2026-07-26T21:40:00Z"  
}
```

List endpoints wrap their items one level deeper in a pagination object:

```
// data: PaginatedResponse<T>  
{  
  "items": [ /* T[] */ ],  
  "totalCount": 42,  
  "pageNumber": 1,  
  "pageSize": 20,  
  "totalPages": 3,  
  "hasNextPage": true,  
  "hasPreviousPage": false  
}
```

03 Guest endpoints — User Portal / mobile app

Base: `/api/v1/support-chat/my` . Auth: guest's own Bearer token (from OTP login). GuestId is read from the token — never passed by the client.

GET `/api/v1/support-chat/my/conversations`

Purpose: Guest's own conversation summary — call this to show the chat's header state (Open/Closed, unread badge, last message preview) before opening the thread.

No params

Returns a **list** (0 or 1 item today)

FIELD	TYPE	NOTES
<code>id</code>	guid	the conversationId — not used by the guest client, but useful for logging
<code>status</code>	string	"Open" or "Closed"
<code>lastMessagePreview</code>	string	first ~200 chars of the last message
<code>lastMessageAt</code>	datetime?	null if no messages yet
<code>lastMessageFromGuest</code>	bool	true = guest sent the last message
<code>unreadCount</code>	int	unread admin replies, from the guest's perspective

GET /api/v1/support-chat/my/messages?pageNumber=1&pageSize=50

Purpose: Full message history for the chat screen. Returns oldest→newest within the returned page (standard chat order).

Query: **pageNumber** (default 1)

Query: **pageSize** (default 50)

FIELD	TYPE	NOTES
id	guid	message id
body	string	message text
fromGuest	bool	true = guest's own bubble, false = admin's
sentAt	datetime	timestamp
isRead	bool	read by the receiving side
readAt	datetime?	read receipt timestamp
senderName	string?	set only on admin-sent messages (e.g. "System Administrator")
attachmentUrl / attachmentType	string?	reserved for future file attachments — always null today

POST /api/v1/support-chat/my/messages

Purpose: Guest sends a new message to support. Creates the conversation on first use; reopens it automatically if it was closed.

Rate limited: 20/min

```
// request body
{ "body": "My badge isn't working at the entrance" }
```

Response **data** is a single message object — same shape as the list above.

POST /api/v1/support-chat/my/messages/read

Purpose: Guest marks all admin replies as read — call when the guest opens the chat screen.

No body

data: true

04 Admin endpoints — Admin Portal

Base: `/api/v1/support-chat`. Auth: admin's Bearer token, plus a permission — `SupportChat.View` for reads, `SupportChat.Manage` for actions. The current admin already has both.

GET `/api/v1/support-chat/conversations?onlyUnread=false&status=&pageNumber=1&pageSize=20&searchTerm=`

Purpose: The inbox list — every guest who has messaged support. **Sorted unread-first**, then by most recent activity, so this is the source for the "who needs a reply" tab.

Permission: `SupportChat.View`

Query: `onlyUnread` — true to show only guests with unread messages

Query: `status` — "Open" / "Closed" filter

Query: `searchTerm` — matches guest name/email

FIELD	TYPE	NOTES
<code>id</code>	guid	this is the conversationId — pass it to every endpoint below
<code>guestId</code>	guid	the guest's public id
<code>guestName</code> / <code>guestEmail</code>	string	for the list row
<code>status</code>	string	"Open" / "Closed"
<code>lastMessagePreview</code>	string	row subtitle
<code>lastMessageAt</code>	datetime?	row timestamp
<code>lastMessageFromGuest</code>	bool	whose turn it is to reply
<code>unreadCount</code>	int	unread guest messages — drive the unread badge from this

GET `/api/v1/support-chat/conversations/{conversationId}/messages?pageNumber=1&pageSize=50`

Purpose: Full chat history for one guest, after the admin selects them from the inbox list.

Permission: `SupportChat.View`

Same message shape as the guest's `GET /support/messages` above.

POST /api/v1/support-chat/conversations/{conversationId}/messages

Purpose: Admin replies to the guest.

Permission: **SupportChat.Manage**

Rate limited: 20/min

```
// request body
{ "body": "Thanks for flagging – reissuing your badge now." }
```

POST /api/v1/support-chat/conversations/{conversationId}/read

Purpose: Admin marks the guest's messages as read — call when the admin opens a conversation. This is what clears the unread badge in the inbox list.

Permission: **SupportChat.Manage**

No body

POST /api/v1/support-chat/conversations/{conversationId}/close

Purpose: Marks the ticket resolved. If the guest sends another message later, it reopens automatically — no extra call needed.

Permission: **SupportChat.Manage**

No body

POST /api/v1/support-chat/conversations/{conversationId}/reopen

Purpose: Manually reopen a closed conversation.

Permission: **SupportChat.Manage**

No body

05 All endpoints, at a glance

WHO	METHOD & PATH	DOES
Guest	GET /support-chat/my/conversations	My conversation summary
Guest	GET /support-chat/my/messages	My message history (paged)
Guest	POST /support-chat/my/messages	Send a message
Guest	POST /support-chat/my/messages/read	Mark admin replies as read
Admin	GET /support-chat/conversations	Inbox list (unread-first)
Admin	GET /support-chat/conversations/{id}/messages	One guest's full history
Admin	POST /support-chat/conversations/{id}/messages	Reply to guest
Admin	POST /support-chat/conversations/{id}/read	Mark guest's messages as read
Admin	POST /support-chat/conversations/{id}/close	Resolve the ticket
Admin	POST /support-chat/conversations/{id}/reopen	Reopen a resolved ticket

06 Real-time & notifications

The frontend can optionally listen for live updates instead of polling.

- A SignalR hub is already mounted at `/realtimehub`. Connect with the same Bearer token used for the REST calls.
- When a new message arrives, the server pushes an event named `support-message-new` to the recipient's connection — admin connections receive it directly; guest connections receive it via a `guest:{guestId}` group.
- Treat it as a "refetch the conversation/messages" signal — the payload carries the `conversationId` but the REST endpoints above remain the source of truth.
- Every message also creates a normal notification row (visible via the existing admin notification bell / guest notification bell endpoints), so nothing is missed even if the socket disconnects.

Push notifications (mobile lock-screen style) are not live yet — only the in-app real-time channel above works today. The backend is structured so real push can be added later without changing any of the endpoints in this document.

